

Entrega e implantação contínuas I

Semana 6

CSDV-246 - DevOps



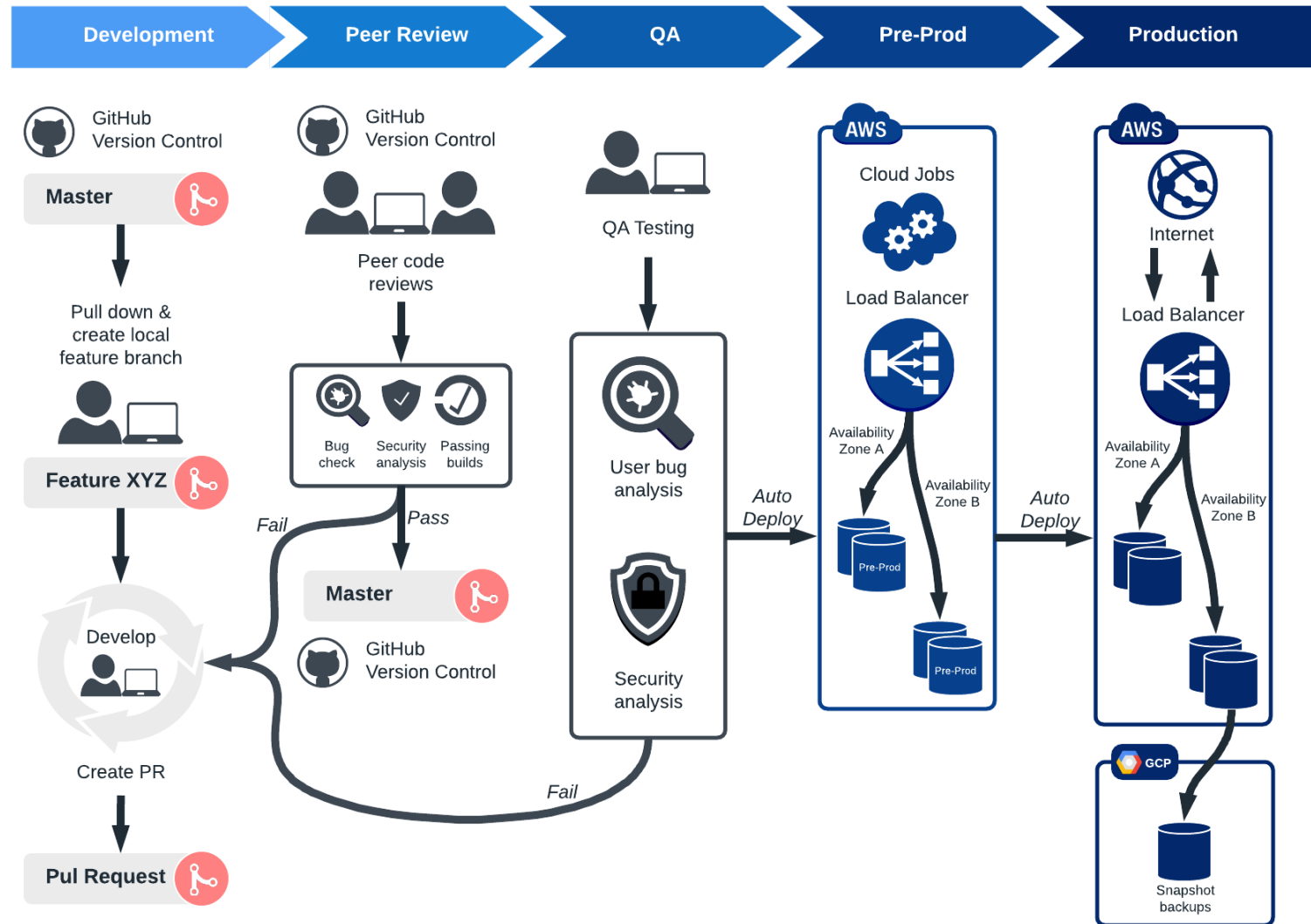
Conteúdo

- O que é entrega contínua.
- O que é implantação contínua.
- Diferenças entre entrega contínua e implantação.
- Noções básicas sobre Script Bash.
- Teste de aceitação
- Estratégias de implantação
- Implantar Recriar.
- Implantação azul-verde.

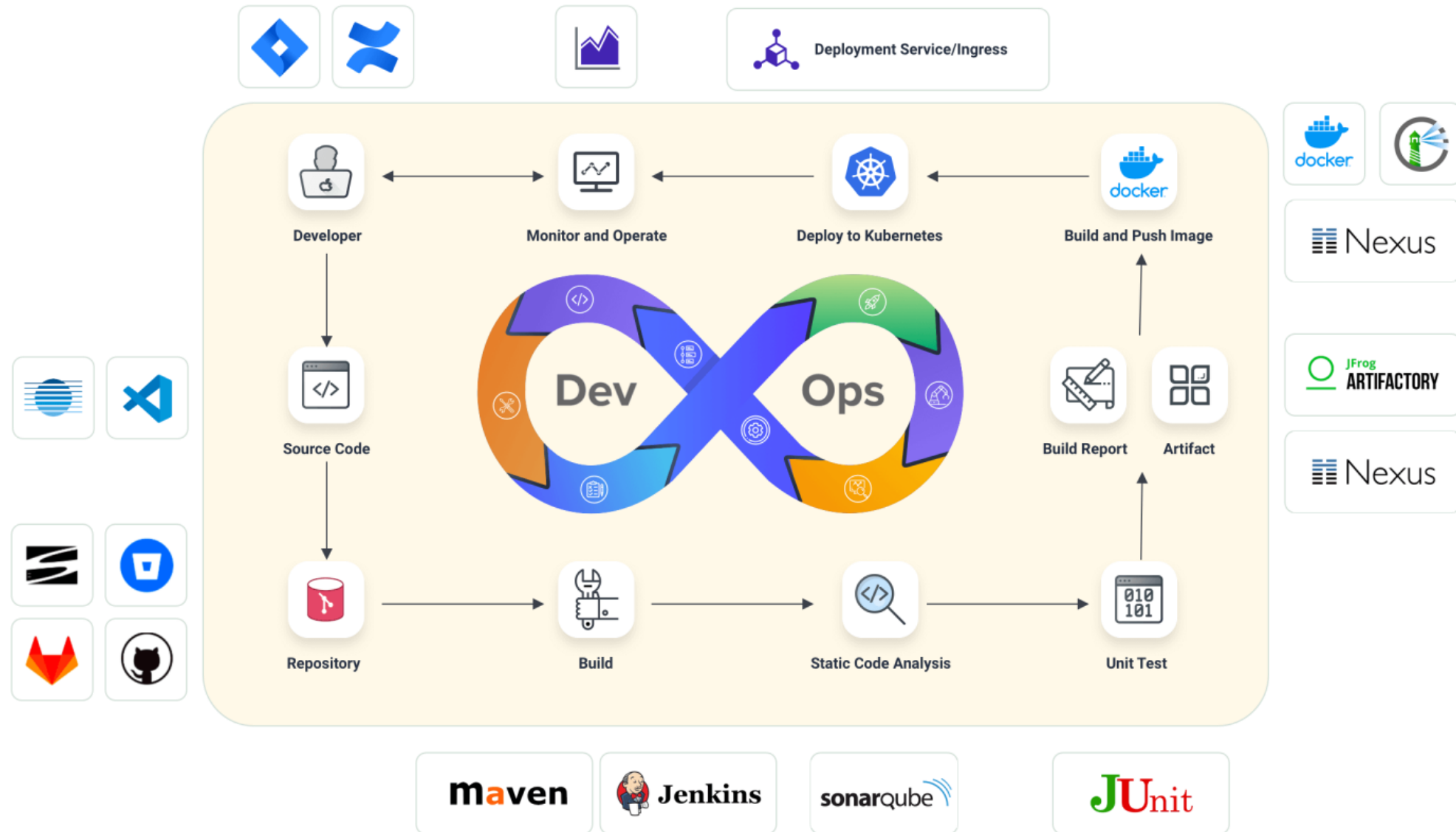


O que é entrega contínua?

O que é entrega contínua?



O que é entrega contínua?



Entrega continua

Entrega contínua



"A entrega contínua é uma prática de desenvolvimento de software em que as alterações no código são automaticamente compiladas, testadas, preparadas e entregues à produção. Estende a integração contínua implantando todas as alterações de código em um ambiente de teste ou produção após a fase de compilação. Quando a entrega contínua é implementada corretamente, os desenvolvedores sempre terão um artefato pronto para implantação que passou por um processo de teste padronizado." Fonte: AWS

Entrega contínua



- A Entrega Contínua (CD) é a próxima etapa lógica da Integração Contínua.
- As alterações de código são criadas, testadas e empacotadas automaticamente para liberação para produção.
- O objetivo é lançar atualizações para os usuários de forma rápida e sustentável.

Entrega contínua



Benefícios

- A entrega contínua permite que a equipe crie, teste e prepare automaticamente as alterações de código para implantação na produção, melhorando a eficiência e a velocidade da entrega de software.
- Essas práticas melhoram a produtividade da equipe, liberando os desenvolvedores de tarefas manuais, ajudando a reduzir o número de erros e bugs que podem chegar à produção.

Entrega contínua



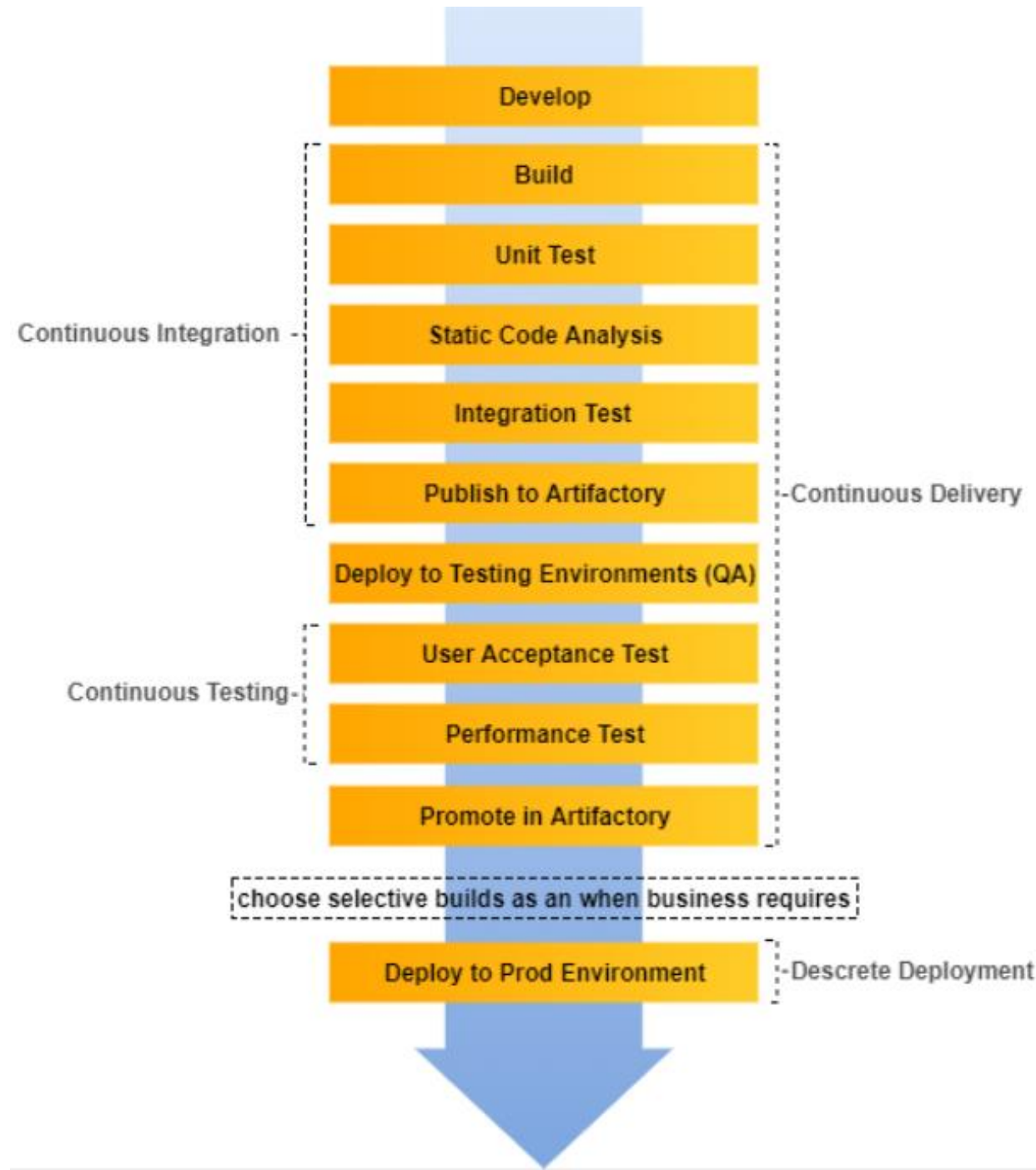
Benefícios

- A Entrega Contínua permite tipos adicionais de testes no código, pois todo o processo seria automatizado.
- A entrega contínua permite que as atualizações sejam entregues aos clientes com mais rapidez e frequência, graças a artefatos que estariam prontos para implantação depois de passarem por um processo de teste padronizado.

Entrega contínua

Workflow

- A entrega contínua começa com a integração contínua.
- O artefato gerado pela Integração Contínua será submetido a rigorosos testes automatizados.



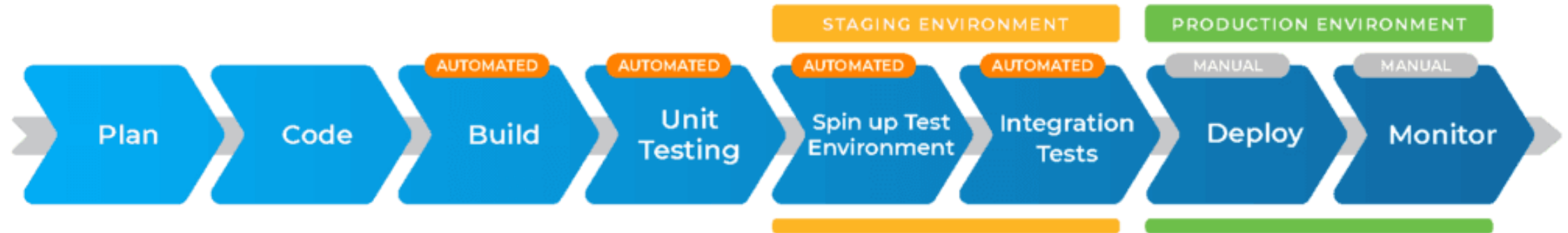
Entrega contínua

Workflow

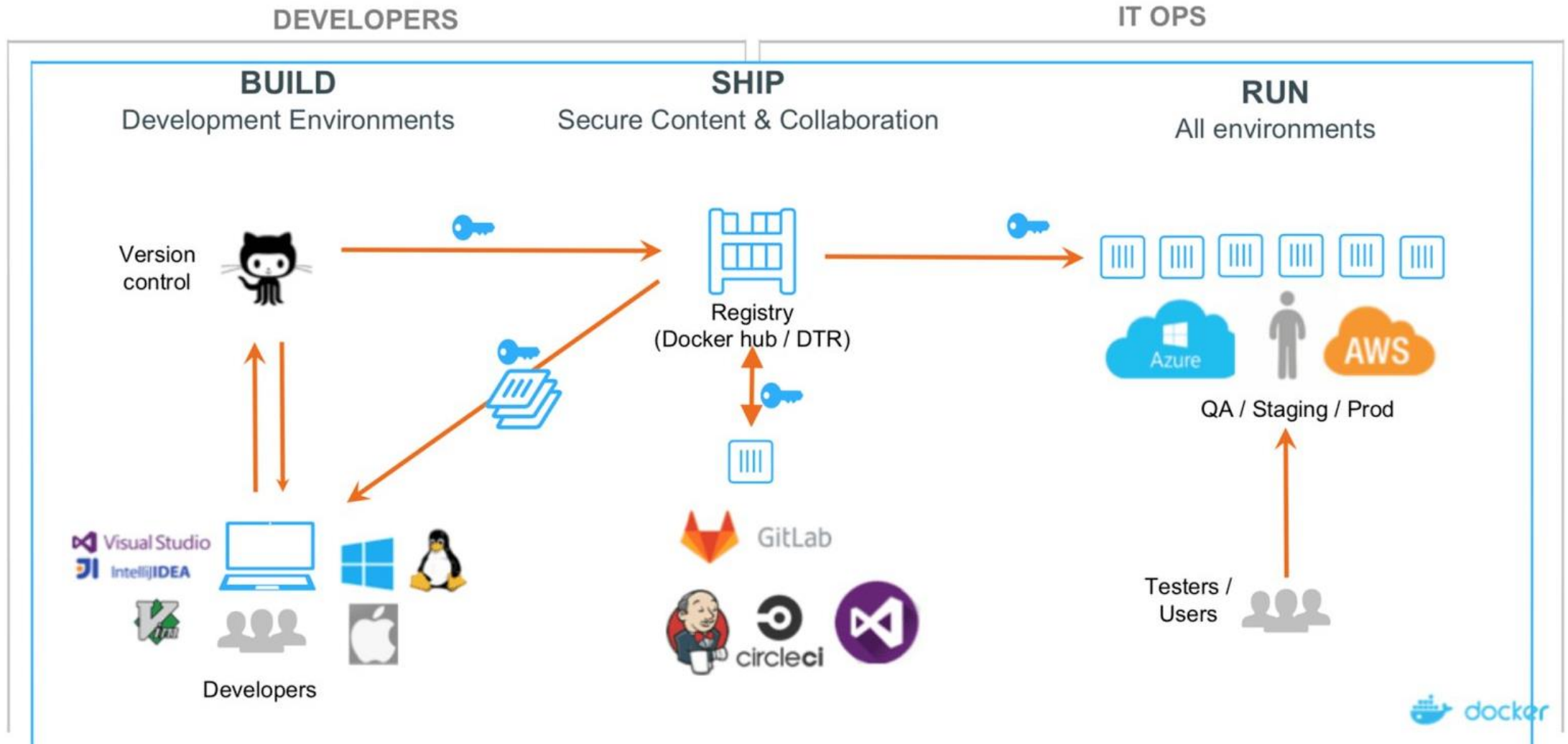
- A Entrega Contínua termina quando obtemos uma versão do artefato que passou com sucesso em todos os testes automatizados, sendo considerada como um artefato pronto para produção.
- Esse artefato é marcado e armazenado nos Repositórios Binários para distribuição e/ou implantação.



Entrega contínua



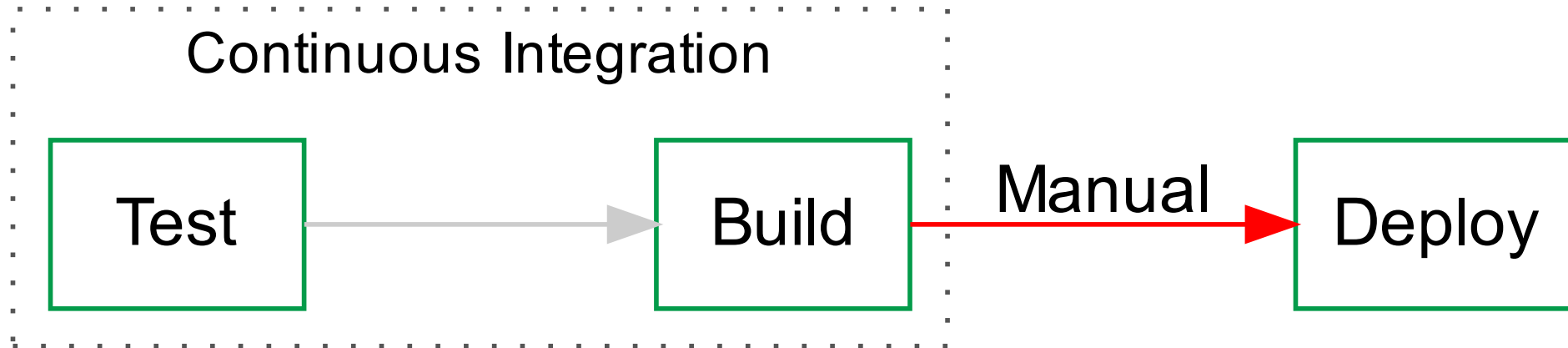
Entrega contínua



Entrega contínua



Continuous Delivery



Implantação contínua

Implantação contínua



"A fase final do pipeline de implantação de CI/CD é a implantação contínua, que pode incluir a automação completa de todo o processo de lançamento de software, incluindo a implantação no ambiente de produção. Em um ambiente de CI/CD totalmente maduro, o caminho para o ambiente de produção é totalmente automatizado, permitindo que o código seja implantado com alta confiança."

Fonte: AWS

Implantação contínua



- A Implantação Contínua é uma extensão da Entrega Contínua.
- A ideia é pegar a nova versão estável do software que obtemos por meio da Entrega Contínua e implantá-la em um ambiente de Produção sem intervenção humana.
- Em outras palavras, todas as alterações no Software que foram validadas como estáveis são publicadas automaticamente para os usuários finais/clientes.

Implantação contínua

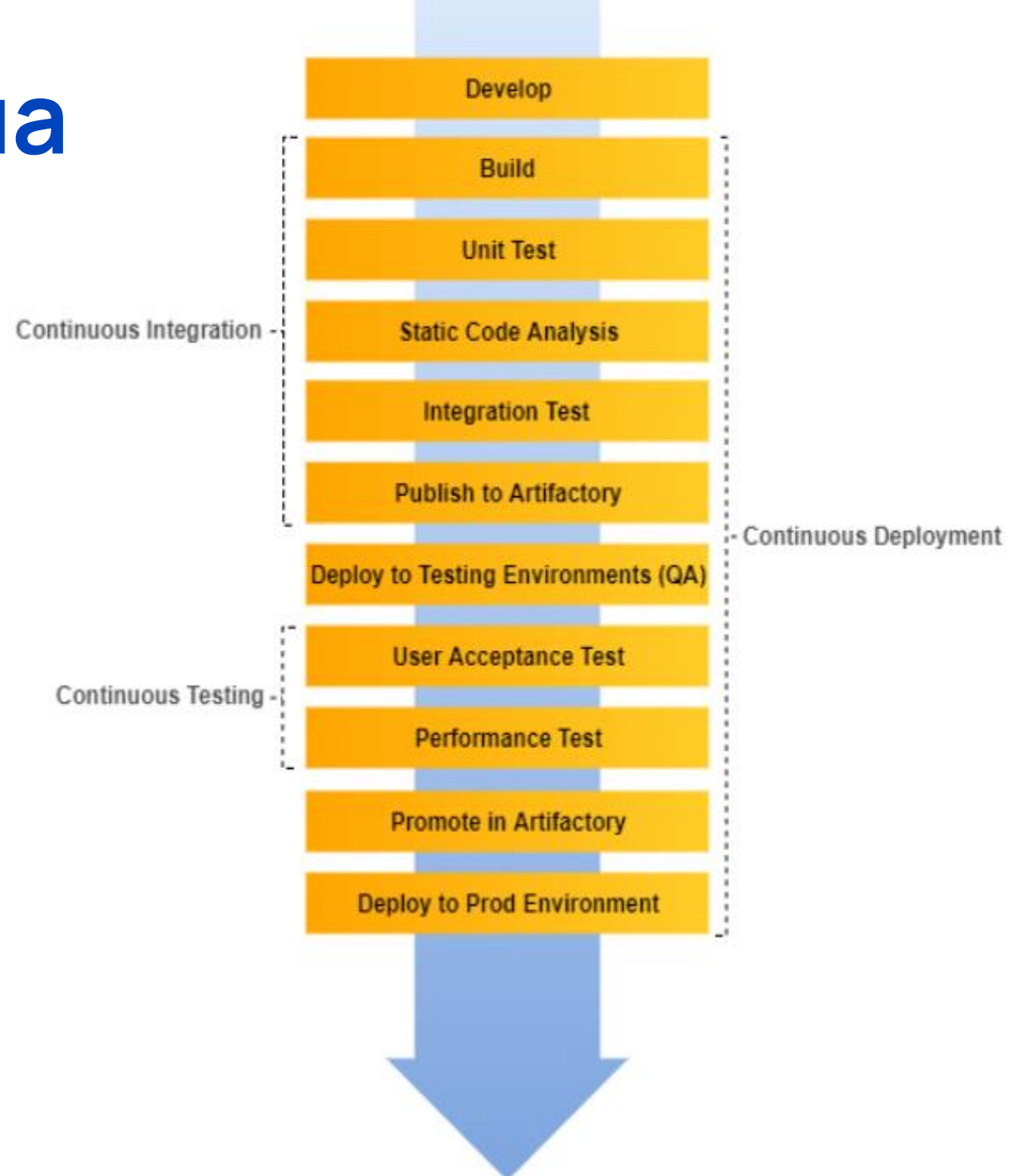


- Sem a implantação contínua, as implantações no ambiente de produção são feitas manualmente.
- São seguidos manuais de implantação, que descrevem o passo a passo que os operadores devem executar para que a implantação seja bem-sucedida.
- As datas e horários de interrupção do serviço estão programados para implantação.
- Geralmente, essas implantações manuais acontecem tarde da noite, quando o tráfego no serviço é mínimo, tentando prejudicar o mínimo possível os usuários finais.

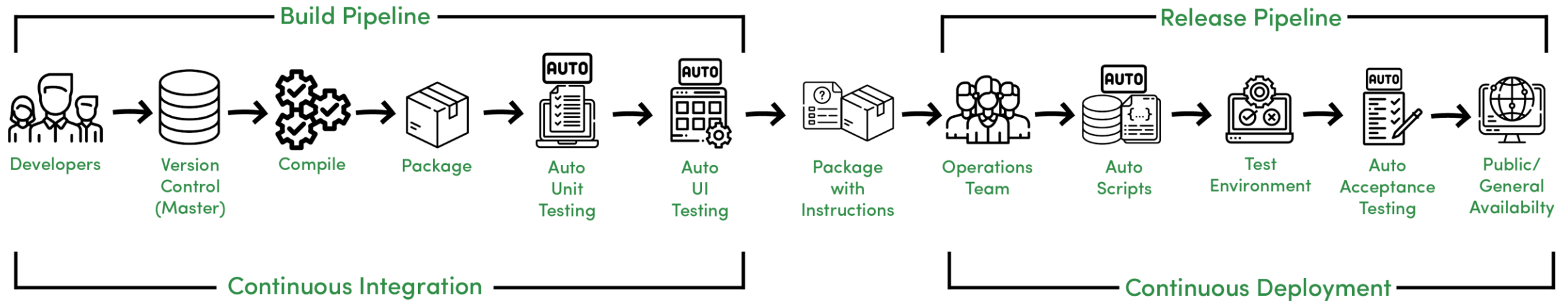
Implantação contínua

Workflow

- A implantação contínua é composta por integração contínua e entrega contínua.
- O fluxo começa com a compilação do novo recurso do software e termina quando o recurso é implantado automaticamente e com sucesso no ambiente de produção.



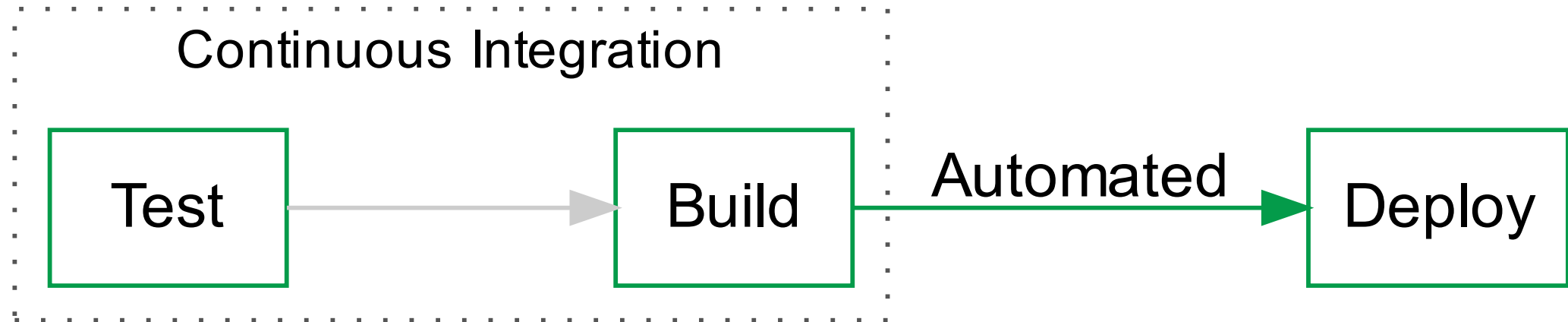
Implantação contínua



Continuous Deployment



Continuous Deployment



Diferencias entre Entrega contínua e Implantação contínua.

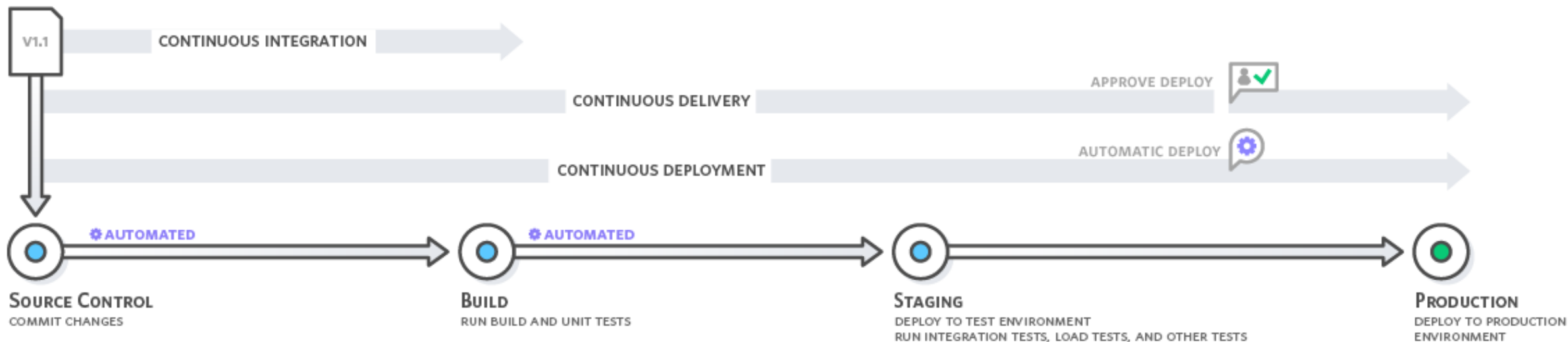
Entrega x Implantação



A principal diferença entre Entrega Contínua e Implantação Contínua é baseada na decisão de atualizar o ambiente de Produção.

- Se a decisão envolver a atualização manual do ambiente de produção, estaremos diante de um fluxo de trabalho que corresponde à Entrega contínua.
- Se a decisão de atualizar o ambiente de produção for tomada automaticamente, sem aprovação explícita, será um fluxo de trabalho de implantação contínua.

Entrega x Implantação



Fuente: AWS

Noções sobre Script Bash

Script Bash



Introdução

- Os scripts Bash são ótimos para automatizar cargas de trabalho repetitivas e economizar tempo.
- Bash é um interpretador de shell Unix.
- Está amplamente disponível em vários sistemas operacionais e também é o shell padrão na maioria dos sistemas operacionais Linux.

Script Bash



Introdução

- Por exemplo, em um grupo de sete desenvolvedores em um projeto que requer uma configuração de ambiente um pouco demorada, cada desenvolvedor precisa configurar o ambiente manualmente. Essa tarefa será repetida sete vezes.
- Em vez de fazer o mesmo trabalho repetitivo, você pode criar um arquivo de texto simples contendo todas as instruções necessárias e enviá-lo para toda a equipe.
- O que resta a fazer é executar o script Bash e tudo será criado/configurado automaticamente.

Script Bash



Criação

- Primeiro, um arquivo com uma extensão .sh deve ser criado:
`$ touch meu-script.sh`
- A primeira linha do arquivo deve ter informações sobre o interpretador de comandos a ser usado, pode ser /bin/bash ou /bin/sh. Isso ocorre por causa das convenções de código:

```
#!/bin/bash
```

Script Bash



Estrutura

- Em seguida, adicionamos o clássico "Hello World!" e salvamos o arquivo:

```
#!/bin/bash  
echo "Hello World!"
```

- Agora precisamos tornar o script um arquivo executável:

```
$ chmod +x meu-script.sh
```

Script Bash



Execução

- Agora podemos executá-lo com `./`

```
$ ./meu-script.sh
```

- E obteremos o seguinte no terminal:
Hello World!

Script Bash



Variáveis

- Para atribuir valores a uma variável, basta usar o =
- Em um terminal Linux, poderíamos criar a variável CITY e atribuir a ela o valor Brooklyn, esta variável é temporária:

```
$ CITY="Brooklyn"
```

- Para imprimir o valor da variável criada na tela, usamos o comando echo:

```
$ echo $CITY
```

Script Bash



Variáveis

- Criando variáveis no script:

```
#!/bin/bash  
CITY="SAN FRANCISCO!"  
echo "OLA $CITY"
```

- Depois de executar o script, obteremos o seguinte no terminal:

```
$ ./script.sh  
OLA SAN FRANCISCO!
```

Script Bash



Ler entrada do teclado

- Também podemos fazer com que o script leia uma entrada inserida pelo usuário do teclado usando o comando "read".

```
#!/bin/bash
echo "Qual seu nome?"
read NAME
echo "Hey! $NAME"
echo "Bem vindo ao DevOps!"
```

Script Bash



Comentários

- Os comentários em um script são importantes para qualquer membro da equipe entender do que se trata determinados segmentos de código, é para isso que o símbolo é usado "#".

```
#!/bin/bash

# Perguntamos ao usuário seu nome
echo "Qual é seu nome?"

# Salvamos os dados inseridos pelo teclado em NAME
read NAME

# Imprimimos uma mensagem de boas-vindas na tela
echo "Hey! $NAME"
echo "Bem vindo ao DevOps!"
```

Script Bash



Argumentos

- Outro elemento importante no script é o tratamento de argumentos, para isso as variáveis **\$1**, **\$2**, **\$3** e assim por diante são utilizadas de acordo com a quantidade de dados que o script pode

```
#!/bin/bash
```

```
echo "O primeiro parâmetro é: $1"
```

```
echo "O segundo parâmetro é: $2"
```

```
echo "O terceiro parâmetro é: $3"
```

Passa valores para parâmetros do terminal:

```
$ ./script.sh "DevOps" "CI" "CD"
```




Script Bash - if

Condicional "if"

- O formato da condicional "if" é o seguinte:

```
#!/bin/bash
# Exemplo com if
echo "Qual seu nome?"
read NAME
if [[ -z ${NAME} ]]
then
    echo "Por favor, digite seu nome!"
fi
```

Script Bash



Condicional "if-else"

```
#!/bin/bash

# Exemplo com if
AdminUser="jorge.lopez"
echo "Qual sua conta de usuario?"
read UserName

# Verificamos se o nome de usuário inserido é o administrador do
host.
if [[ ${UserName} == ${AdminName} ]]; then
    echo "Você é o administrador $AdminUser deste host."
else
    echo "Você não é o administrador."
fi
```

Script Bash



Condicional "switch-case"

```
#!/bin/bash
case "$variable" in
    padrao1)
        # qualquer comando
        echo "Exemplo: $variable"
        ;;
    padrao2 | padrao3)
        # Qualquer comando
        echo "Exemplo: $variable"
        ;;
    *)
        echo "Sem padrão"
        ;;
esac
```

Script Bash



Exemplo "switch-case"

```
#!/bin/bash
read -p "Digita a marca do seu carro: " CAR
case "$CAR" in
    Tesla)
        echo -n "A fábrica de $CAR está nos EUA."
        ;;
    BMW | Mercedes | Audi | Porsche)
        echo -n "A fábrica de $CAR está na Alemanha."
        ;;
    *)
        echo -n "A marca $CAR não é conhecida."
        ;;
esac
```

Script Bash



Laço "for"

```
#!/bin/bash  
  
for var in ${list}  
do  
    # comandos  
done
```

```
#!/bin/bash  
  
USUARIOS="admin richard kerstin"  
  
for user in ${USUARIOS}  
do  
    echo "$user"  
done
```

Script Bash



Laço "while"

```
#!/bin/bash  
  
while [condição]  
do  
    # comandos  
done
```

```
#!/bin/bash  
contador="1"  
  
while [[ ${USUARIOS} -le 10]]  
do  
    echo "$contador"  
    ((contador++))  
done
```

Script Bash



Continue

- Isso significa que dizemos ao script para interromper a iteração atual e iniciar a próxima iteração:

```
continue [n]
```

- Observe que [n] é opcional.

Script Bash



Continue

```
#!/bin/bash
for i in 1 2 3 4 5
do
    if [[ $i -eq 2 ]]; then
        echo "Pulamos a iteração2"
        continue
    fi
    echo "$i é igual a $i"
done
```


Script Bash



Break

- Isso significa que dizemos ao script para encerrar o(s) ciclo(s) atual(is), for ou while, sintaxe:

```
break [n]
```

- Observe que [n] é opcional.

Script Bash



Break

```
#!/bin/bash
num=1
while [[ $num -lt 10 ]]
do
    if [ -n "$num" ] && [ "$num" -eq 5 ]; then
        break
    fi
    ((num++))
done
echo "Loop concluído em $num"
```

Script Bash



Functions

- As funções são uma boa opção para reutilizar código, sintaxe:

```
function nome_funcao() {  
    # Commands  
}
```

Script Bash



Functions

```
#!/bin/bash

function saudacao() {
    echo "A funcao te diz oi!"
}

# Invocamos a função pelo seu nome
saudacao
```

Script Bash



Funções com parâmetros

- Você também pode passar parâmetros para uma função, exemplo:

```
function saudacao() {  
    echo "Ola $1, a funcao le diz oi!"  
}  
# Invocamos a função pelo seu nome  
saludo "estudantes"
```

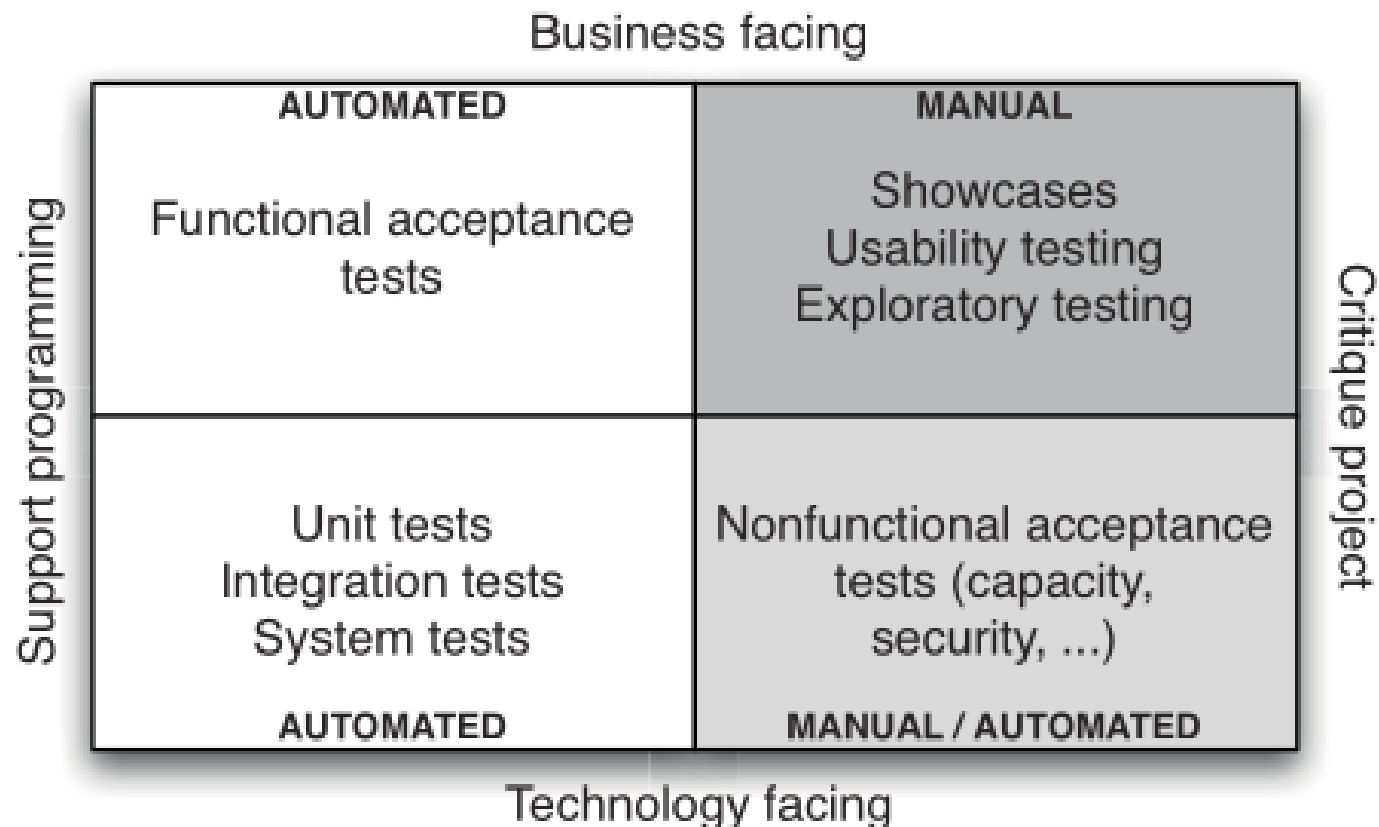
Teste de aceitação

Teste de aceitação



Tipos de testes

- Existem diferentes tipos de testes, este gráfico mostra como eles podem ser organizados:



Teste de aceitação



- O Teste de Aceitação também é conhecido como Teste de Aceitação do Usuário (UAT).
- É um processo de controle de qualidade que é feito para verificar se um sistema atende aos requisitos dos usuários e da empresa.
- Esses testes de aceitação geralmente são realizados nos estágios posteriores, antes do lançamento de uma nova versão do software.
- Eles geralmente são realizados em um ambiente muito semelhante à produção e basicamente se concentram em replicar as ações dos usuários.

Teste de aceitação - quem e quando

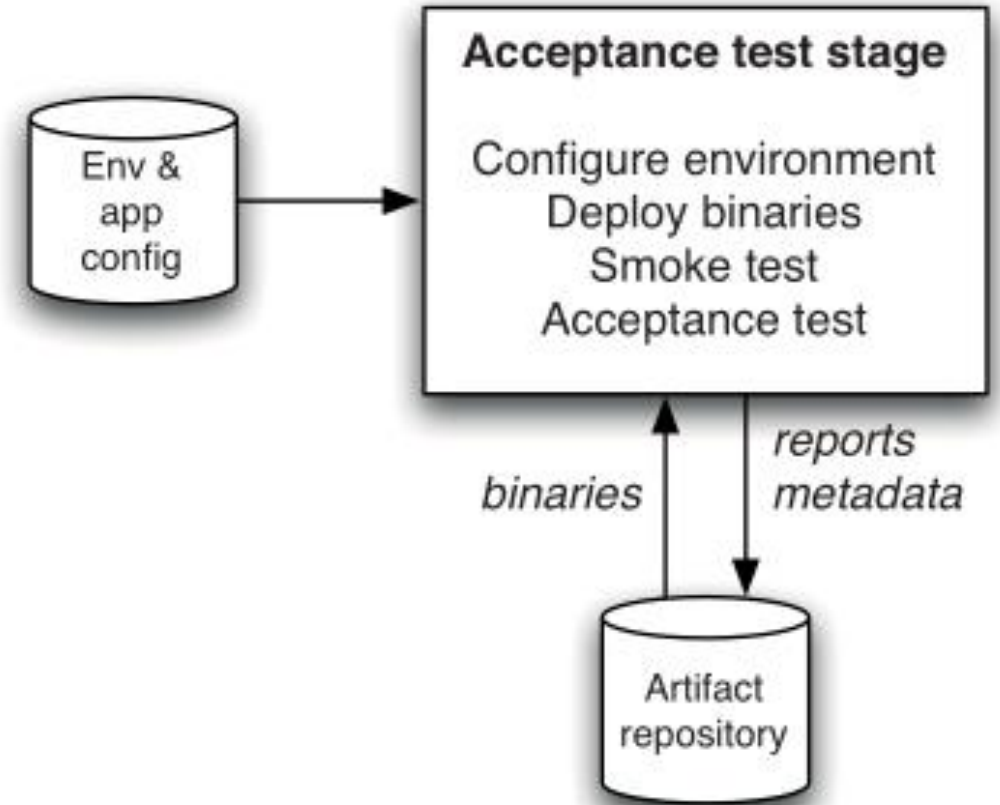
- O teste de aceitação pode ser realizado pela equipe de desenvolvimento ou controle de qualidade, e o teste também pode ser realizado pelos próprios clientes finais.
- Desta forma, garante-se que o software cumpre as características e comportamentos esperados e/ou exigidos.
- É uma boa maneira de evitar o lançamento de produtos de software incompletos ou defeituosos.



Teste de aceitação



- Esses testes geralmente são executados para cada versão do software.
- O fluxo de trabalho do estágio Teste de aceitação no pipeline de entrega é explicado no diagrama.



Teste de aceitação



Steps in the UAT process

ONE



Plan

TWO



Identify and
create real-world
test scenarios

THREE



Select the
testing team

FOUR



Test and
document

FIVE



Update code,
retest and
sign off

ICONS: MAKSIM ANKUDA/GETTY IMAGES (1,2,3,4), PEACEFULLY7/GETTY IMAGES (5)

©2022 TECHTARGET. ALL RIGHTS RESERVED 

Fuente: TechTarget

Estratégias de implantação

Estratégias de implantação



- Uma estratégia de implantação pode ser entendida como uma abordagem planejada para fornecer novas versões do software aos usuários finais.
- As implantações têm um alto risco de falha, especialmente se as implantações forem feitas no ambiente de produção.
- As estratégias de implantação buscam garantir a estabilidade do sistema e mitigar o risco de falhas.

Estratégias de implantação



- Cada estratégia determina as técnicas de como a mudança, atualização, reversão, etc. será feita. De uma versão existente do sistema para uma nova, priorizando a experiência do usuário e a qualidade do serviço.
- Tais estratégias permitem:
 - Minimize o tempo de inatividade.
 - Reduza o impacto de erros na produção.
 - Garanta um retorno rápido às versões estáveis se surgir um problema.

Estratégias de implantação



- Existem várias estratégias, algumas das quais resultam em tempo de inatividade durante o processo de atualização.
- Alguns permitem que o usuário não tenha conhecimento do desempenho de uma implantação, ou seja, são transparentes para o usuário.
- Alguns são mais eficientes, alguns são mais baratos, alguns são mais complexos de executar, etc.

Estratégias de implantação



Lista de estratégias de implantação

- **Basic Deployment**
Atualização direta do sistema com tempo de inatividade.
- **Recreate Deployment**
Pare e substitua completamente o aplicativo.
- **Rolling Deployment**
Atualização gradual dos serviços sem interrupção total.

Estratégias de implantação



Lista de estratégias de implantação

- **Blue-Green Deployment**
Manter dois ambientes, um ativo e outro pronto como backup.
- **Canary Deployment**
Libere gradualmente as alterações para um subconjunto de usuários.
- **Feature Flags**
Habilitar recursos com base nas configurações de tempo de execução.

Estratégias de implantação



Lista de estratégias de implantação

- **A/B Testing**

Teste diferentes versões para avaliar a resposta do usuário.

- **Shadow Deployment**

Execute novas versões em paralelo para testar sem afetar o usuário.

- **Progressive Deployment**

Aumente gradualmente a exposição de novos softwares com monitoramento contínuo.

Estratégias de implantação



Recreate Deployment

- A estratégia de recriação envolve encerrar todas as instâncias existentes antes de criar novas.
- Isso gera logicamente tempo de inatividade durante o processo de atualização, mas garante uma transição limpa e esperada.
- Essa estratégia é recomendada para os seguintes casos:

Estratégias de implantação



Recriar casos de uso de implantação

- **Sistemas não críticos:** quando o sistema pode suportar um tempo de inatividade temporário sem afetar significativamente a experiência do usuário ou as operações de negócios.
- **Baixas frequências** de implantação: se as alterações de software forem pouco frequentes, essa estratégia pode ser viável minimizando a complexidade.
- **Ambientes simples:** em sistemas com poucos componentes ou dependências, onde a substituição completa é mais fácil de gerenciar.

Estratégias de implantação



Casos de Uso de Recreate Deployment

- **Entornos controlados:** Como preproducción o desarrollo, donde el tiempo de inactividad no impacta a los usuarios finales.

Estratégias de implantação



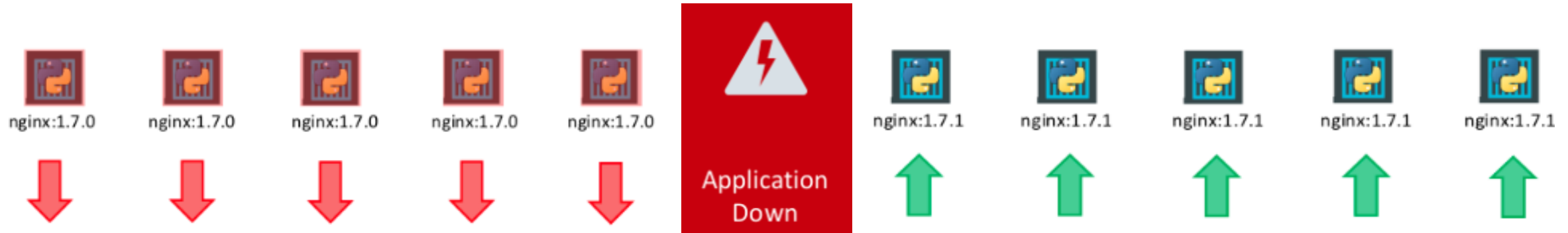
Workflow de Recreate Deployment

- Este processo envolve o seguinte:
 - **Parar o serviço atual:** O aplicativo existente é desativado ou removido do ambiente.
 - **Substituição completa:** A nova versão do software é implantada do zero, substituindo a anterior.
 - **Início do novo sistema:** O serviço atualizado começa a operar, assumindo o controle do ambiente.

Estratégias de implantação



Workflow de Recreate Deployment



Estratégias de implantação



Blue/Green Deployment

- Consiste na manutenção de dois ambientes idênticos, Azul (produção atual) e Verde (nova versão).
- O tráfego do usuário é inicialmente direcionado para o ambiente azul, enquanto o verde é configurado e testado.
- Depois que o Green é verificado como estável, todo o tráfego é redirecionado para esse ambiente, tornando-se a nova "produção".
- Se houver problemas, o tráfego pode ser rapidamente redirecionado de volta para o ambiente Blue, minimizando o impacto.

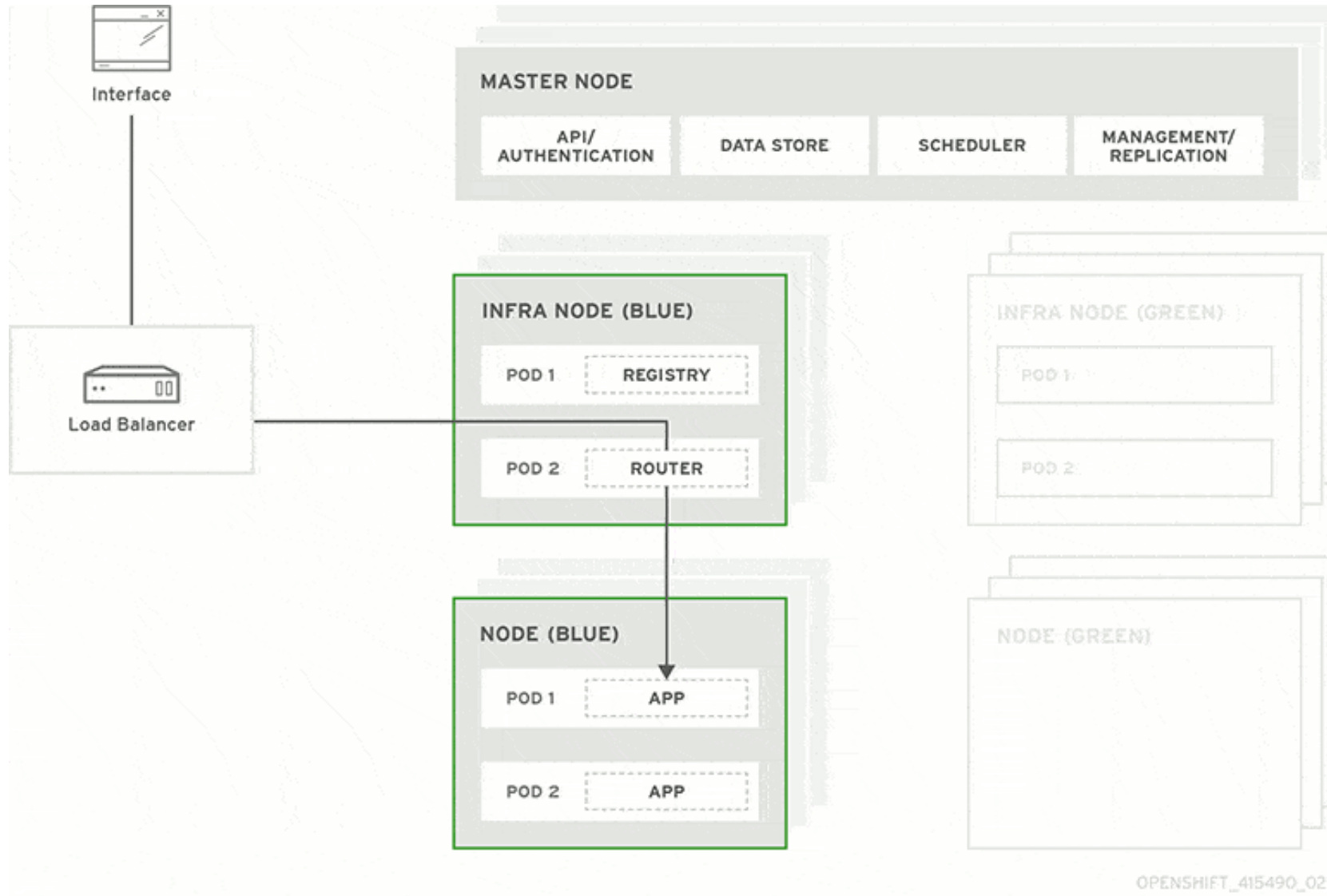
Estratégias de implantação



Casos de Uso de Blue/Green Deployment

- **Sistemas críticos:** Quando o tempo de inatividade precisa ser minimizado e garantir uma experiência perfeita para os usuários finais.
- **Implantações frequentes:** Em ambientes onde as alterações de software são frequentes e devem ser realizadas com baixo risco.
- **Testes em produção:** Se é importante validar a nova versão em um ambiente real antes de redirecionar o tráfego do usuário.
- **Reversão rápida:** Quando a capacidade de reverter imediatamente para uma versão anterior é necessária se ocorrer um problema.

Estratégias de implantação



Q&A